

Bootstrap Confidence Intervals, Worked Out by Hand

Resampling the sample to put error bars on a metric

A single test-set number — “F1 = 0.756” — tells you nothing about how much that number would wobble on a different draw of test data. The *percentile bootstrap* answers exactly that, with no normality assumption and no closed-form variance: it treats your one sample *as if it were the population*, draws many resamples *with replacement*, recomputes the metric on each, and reads the confidence interval straight off the percentiles of those values. Every number below was produced by the NumPy reference program in Appendix A, so the arithmetic is exact and reproducible from the printed seed.

What this document does. It takes a tiny sample of $n = 8$ per-fold scores, $x = [0.7, 0.8, 0.6, 0.9, 0.75, 0.85, 0.65, 0.8]$, and builds a 95% confidence interval for its mean entirely by hand: four explicit seeded resamples computed digit-by-digit, then the full B -replicate sampling distribution and the 2.5th / 97.5th percentile bounds that bracket it.

How to read this. Each step isolates one mechanism — the resample, the replicate statistic, the percentile rule — and gives it a worked box on the cleanest numbers. Read this to understand *why* the bootstrap works and *where* the interval comes from; the course’s `bootstrap_ci` helper is just this loop wrapped in a function.

Concepts covered, in order: the sample and its point estimate · one resample with replacement · the replicate statistic (mean) · four seeded replicates, reproduced exactly · the sampling distribution of B replicates · the $\alpha = (1 - c)/2$ percentile rule · reading [2.5%, 97.5%] bounds · how B controls the noise in the bounds · when the bootstrap is the right tool.

1 The setup: one sample, one number, no error bar

Suppose a 5-fold cross-validation (or a per-sample score vector) produced these $n = 8$ values — think of them as F1 scores on 8 folds:

$$x = [0.7, 0.8, 0.6, 0.9, 0.75, 0.85, 0.65, 0.8], \quad n = 8.$$

The point estimate is the sample mean,

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{0.7 + 0.8 + 0.6 + 0.9 + 0.75 + 0.85 + 0.65 + 0.8}{8} = \frac{6.05}{8} = \boxed{0.7563}.$$

That single number is what gets reported. But “0.7563” hides a question every reviewer asks: *if we had drawn a different test set, how different would this be?* The sample standard deviation $s = 0.1016$ (with $\text{ddof} = 1$) hints at spread, but turning that into an interval normally needs a distributional assumption. The bootstrap sidesteps the assumption entirely.

Quantity	Symbol	Value here
Sample size	n	8
Sorted sample	—	[0.6, 0.65, 0.7, 0.75, 0.8, 0.8, 0.85, 0.9]
Point estimate (mean)	$\bar{x}$	0.7563
Sample std (ddof = 1)	s	0.1016
Statistic of interest	$\hat{\theta}$	the mean
Confidence level	c	0.95
Tail probability	$\alpha = (1 - c)/2$	0.025

Intuition

The bootstrap’s one big idea: **your sample is the best picture of the population you have, so use it as the population.** Instead of drawing fresh test sets from the real world (you can’t — you only have one), you draw fresh “test sets” *from your own sample, with replacement*. Each such resample is a plausible alternate reality of equal size n . Recomputing the metric on many of them traces out how much the metric *would* jiggle under resampling — the sampling distribution — without ever assuming it is normal or knowing a variance formula.

2 Step 1 — One resample with replacement

A **bootstrap resample** draws n indices uniformly from $\{0, 1, \dots, n - 1\}$ *with replacement*, then gathers those entries of x . “With replacement” is the whole trick: the same original value can appear several times, others not at all — that duplication and omission *is* the simulated sampling noise.

Using the course’s generator `np.random.default_rng(42)` and drawing $n = 8$ indices gives, for the first resample,

$$\text{idx}^{(1)} = [0, 6, 5, 3, 3, 6, 0, 5] \implies x[\text{idx}^{(1)}] = [0.7, 0.65, 0.85, 0.9, 0.9, 0.65, 0.7, 0.85].$$

Notice index 3 (value 0.9) appears twice, index 0 (value 0.7) twice, index 6 twice, index 5 twice — and indices 1, 2, 4, 7 never appear. That is a legal, equally-likely alternate sample of size 8.

Mini-example — this operation in isolation

Sampling *without* replacement from $\{0, \dots, 7\}$ would just permute the original 8 values — every resample would have the identical mean 0.7563, giving zero variability and a useless interval. *With* replacement, multiplicities differ, so each resample’s mean differs. The bootstrap lives or dies on `replace=True`.

3 Step 2 — The replicate statistic

For each resample we compute the statistic of interest — here the mean — and call the result a **bootstrap replicate** $\hat{\theta}_b^*$. For resample 1:

$$\hat{\theta}_1^* = \frac{0.7 + 0.65 + 0.85 + 0.9 + 0.9 + 0.65 + 0.7 + 0.85}{8} = \frac{6.2}{8} = \boxed{0.775}.$$

One resample, one number. The replicate 0.775 is slightly above the original 0.7563 because this particular draw happened to over-represent the high scores (0.9 twice, 0.85 twice). A different draw will land below. Repeating this produces a *cloud* of replicate values centred near $\bar{x}$.

Watch out

The statistic can be *anything* computable from a resample — median, AUC, F1, a ratio. You do *not* need a variance formula for it. That is the bootstrap’s superpower: the exact same loop that handles the mean handles AUC, where no simple closed-form standard error exists. Just swap the metric function inside the loop.

4 Step 3 — Four seeded replicates, reproduced exactly

To make the procedure fully checkable, here are the first *four* replicates from the same seeded stream `default_rng(42)` (the generator advances between draws, so each row is a fresh resample):

b	resample indices	resampled values	$\hat{\theta}_b^* = \text{mean}$
1	[0, 6, 5, 3, 3, 6, 0, 5]	[0.7, 0.65, 0.85, 0.9, 0.9, 0.65, 0.7, 0.85]	0.7750
2	[1, 0, 4, 7, 5, 6, 5, 6]	[0.8, 0.7, 0.75, 0.8, 0.85, 0.65, 0.85, 0.65]	0.7562
3	[4, 1, 6, 3, 4, 2, 1, 7]	[0.75, 0.8, 0.65, 0.9, 0.75, 0.6, 0.8, 0.8]	0.7563
4	[6, 5, 3, 6, 4, 3, 3, 1]	[0.65, 0.85, 0.9, 0.65, 0.75, 0.9, 0.9, 0.8]	0.8000

Four replicates already span 0.7562 to 0.8000. Their mean is close to $\bar{x}$, and they scatter around it — exactly the wobble we wanted to quantify. The collection $\{\hat{\theta}_1^*, \hat{\theta}_2^*, \hat{\theta}_3^*, \hat{\theta}_4^*, \dots\}$ is the bootstrap sampling distribution of the mean.

Intuition

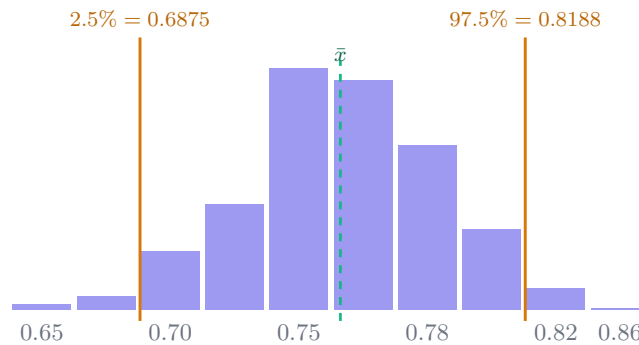
Each replicate is one “what-if the test set had come out slightly differently” trial. Four trials is a rumour; a thousand is a distribution. Stack enough of them and their histogram approximates the true sampling distribution of $\hat{\theta}$ — the thing classical statistics tries to derive with formulas, here obtained by brute-force resampling.

5 Step 4 — The sampling distribution of B replicates

Now repeat the resample-and-recompute loop B times. With $B = 2000$ (seed 42) the 2000 replicate means form a tight, roughly bell-shaped cloud centred on the original estimate:

mean of replicates = 0.7558, std of replicates = 0.0332, min = 0.6437, max = 0.8625.

The std of the replicate means, 0.0332, *is* a bootstrap estimate of the standard error of $\bar{x}$ — computed without the formula $s/\sqrt{n}$ (which here gives $0.1016/\sqrt{8} = 0.0359$, in the same ballpark). The histogram of the 2000 means:



Each bar is the count of replicate means in a 0.022-wide slice; the amber lines are the 2.5th and 97.5th percentiles (next step); the dashed green line is the original estimate. The distribution is centred near $\bar{x} = 0.756$ and tapers symmetrically — this is the sampling variability of the metric made visible.

6 Step 5 — The percentile rule: reading the interval

For a confidence level $c = 0.95$ we want to leave 2.5% of the mass in each tail. The tail probability is

$$\alpha = \frac{1 - c}{2} = \frac{1 - 0.95}{2} = 0.025$$

and the percentile **bootstrap** interval is simply the empirical α and $1 - \alpha$ percentiles of the replicate values:

$$CI_{95\%} = [\hat{\theta}_{(\alpha)}^*, \hat{\theta}_{(1-\alpha)}^*] = [P_{2.5}\{\hat{\theta}_b^*\}, P_{97.5}\{\hat{\theta}_b^*\}].$$

Sort the $B = 2000$ replicate means and look at the value below which 2.5% of them lie, and the value below which 97.5% lie:

$$CI_{95\%} = [0.6875, 0.8188], \quad \text{width} = 0.1312$$

By construction, about 2.5% of replicates (41 of 2000, 2.05%) fall strictly below 0.6875 and about 2.5% (50 of 2000, 2.50%) strictly above 0.8188 — the bounds slice off the tails directly, no z or t multiplier required.

Reporting. You would write: *mean F1 0.756, 95% bootstrap CI [0.688, 0.819] (percentile method, $B = 2000$, seed 42).* The interval is the headline; the width 0.131 says “this metric is known to roughly ± 0.07 .”

When this is the right choice

Reach for the percentile bootstrap when the metric is **non-normal or has no closed-form standard error**: AUC, the median, F1, precision/recall at a threshold, or any ratio. Classical $\bar{x} \pm 1.96 s/\sqrt{n}$ assumes (approximate) normality of the statistic; the bootstrap assumes only that your sample resembles the population. For these awkward metrics it is often the *only* practical interval.

Watch out

Three traps. (1) B **too small** makes the bounds themselves noisy — the percentiles are estimated from few tail values, so they jump between runs. (2) The percentile method is **weak for extreme order statistics** (e.g. the sample maximum), where the resampling distribution is degenerate and the interval can be badly biased — prefer BCa there. (3) Bootstrap results depend on the RNG, so **always report the seed** (here 42) and B , or your “exact” interval is irreproducible.

7 Step 6 — How B controls the noise in the bounds

The replicate count B does *not* change the underlying sampling distribution — it changes how accurately the percentiles estimate it. Small B gives jittery bounds; large B converges. Same sample, same seed, varying B :

B	mean of replicates	std of replicates	95% CI $[P_{2.5}, P_{97.5}]$	width
200	0.7563	0.0316	[0.6938, 0.8187]	0.1250
1000	0.7554	0.0336	[0.6936, 0.8187]	0.1252
2000	0.7558	0.0332	[0.6875, 0.8188]	0.1312
10000	0.7563	0.0332	[0.6875, 0.8188]	0.1313

The lower bound creeps from 0.6938 ($B = 200$) to a stable 0.6875 by $B = 2000$, and the width settles around 0.131. Beyond $B \approx 2000$ the interval barely moves — the extra replicates only polish the tail estimates. The practical rule the course uses is $B \in [1000, 10000]$: cheap enough to run, large enough that the bounds stop wandering.

Intuition

Think of B as the resolution of a photograph of the sampling distribution. The distribution itself is fixed by your 8 data points; B just decides how many pixels you spend imaging its tails. The center stabilises fast (few hundred replicates); the 2.5% tails — estimated from the rarest replicates — need more, which is why small B rattles the *bounds* far more than the *mean*.

8 The procedure, at a glance

#	Step	For our sample	Result
1	point estimate	mean of x	$\bar{x} = 0.7563$
2	resample (with repl.)	idx = [0, 6, 5, 3, 3, 6, 0, 5]	alternate sample
3	replicate statistic	mean of resample ₁	0.7750
4	repeat B times	$B = 2000$, seed 42	cloud of means
5	tail probability	$\alpha = (1 - 0.95)/2$	0.025
6	lower bound	$P_{2.5}$ of replicates	0.6875
7	upper bound	$P_{97.5}$ of replicates	0.8188
8	report	mean, CI, seed, B	0.756 [0.688, 0.819]

9 Equation cheat-sheet

Point estimate	$\hat{\theta} = \bar{x} = \frac{1}{n} \sum_i x_i$
One resample	draw n indices $\sim \text{Uniform}\{0, \dots, n-1\}$ <i>with replacement</i>
Replicate	$\hat{\theta}_b^*$ = statistic on the b -th resample
Bootstrap SE	$\widehat{\text{SE}} = \text{std}(\hat{\theta}_1^*, \dots, \hat{\theta}_B^*)$
Tail probability	$\alpha = (1 - c)/2$ ($c = 0.95 \Rightarrow \alpha = 0.025$)
Percentile CI	$[P_{100\alpha}\{\hat{\theta}^*\}, P_{100(1-\alpha)}\{\hat{\theta}^*\}]$
95% bounds	$[P_{2.5}, P_{97.5}]$
Practical B	$1000 \leq B \leq 10000$ (larger $\Rightarrow$ steadier tails)
Always report	seed and B (results are RNG-dependent)

Appendix A Reference implementation

Every number above is reproducible from the following program. It mirrors the course's `bootstrap_ci` helper: draw indices with `rng.integers`, recompute the metric, take percentiles. Real evaluation runs would feed in `y_true`, `y_prob` and a metric like F1 or AUC; here the statistic is the plain mean so the arithmetic is checkable by hand. A fixed seed makes the run reproducible — in production you fix the seed for reporting but understand the interval reflects RNG draws.

```
import numpy as np

x = np.array([0.7, 0.8, 0.6, 0.9, 0.75, 0.85, 0.65, 0.8]) # n=8 fold scores
n = len(x)
print("sample mean:", round(x.mean(), 4))                # 0.7563

# ---- four explicit seeded resamples (reproducible) ----
rng = np.random.default_rng(42)
for b in range(4):
    idx = rng.integers(0, n, size=n) # sample WITH replacement
    sample = x[idx]
    print(b + 1, idx.tolist(), round(sample.mean(), 4))
# 1 [0, 6, 5, 3, 3, 6, 0, 5] 0.775
# 2 [1, 0, 4, 7, 5, 6, 5, 6] 0.7562
# 3 [4, 1, 6, 3, 4, 2, 1, 7] 0.7563
# 4 [6, 5, 3, 6, 4, 3, 3, 1] 0.8

# ---- full percentile bootstrap CI ----
def bootstrap_ci(x, stat=np.mean, B=2000, confidence=0.95, seed=42):
    rng = np.random.default_rng(seed)
    n = len(x)
    reps = np.array([stat(x[rng.integers(0, n, size=n)]) for _ in range(B)])
    alpha = (1 - confidence) / 2 # 0.025
    lo = np.percentile(reps, 100 * alpha) # 2.5th percentile
    hi = np.percentile(reps, 100 * (1 - alpha)) # 97.5th percentile
    return reps.mean(), lo, hi, reps.std(ddof=1)

mean, lo, hi, se = bootstrap_ci(x, B=2000, seed=42)
print(f"mean={mean:.4f} 95% CI=[{lo:.4f}, {hi:.4f}] width={hi-lo:.4f} SE={se:.4f}")
# mean=0.7558 95% CI=[0.6875, 0.8188] width=0.1312 SE=0.0332

# ---- B sensitivity: bounds steady once B is large enough ----
for B in [200, 1000, 2000, 10000]:
    m, lo, hi, _ = bootstrap_ci(x, B=B, seed=42)
    print(B, round(lo, 4), round(hi, 4), round(hi - lo, 4))
# 200 0.6938 0.8187 0.125
# 1000 0.6936 0.8187 0.1252
# 2000 0.6875 0.8188 0.1312
# 10000 0.6875 0.8188 0.1313
```

— end of document —